# Vision-Based PySuperTuxKart Ice Hockey Agent Design

AI 394D Final Project Report

Spring 2024

**Zanwar Faraj** zanwarf@utexas.edu

**Van Quach** vquach718@utexas.edu

**Khurram Rafique** rafique@utexas.edu

**Shabbir Suterwala** shabbirsuterwala@utexas.edu

## Abstract

*This paper presents the design of an automated, vision-based agent for the PySuperTuxKart (PySTK) game environment. Our objective was to maximize the goals scored by our agent in a two vs. two ice hockey match against AI-controlled opponents. We considered implementing either a vision-based agent, which reconstructs the game state from gameplay images, or a state-based agent, which is provided with the true state of the game. We ultimately implemented a vision-based agent because we were interested in furthering our expertise in computer vision and found the reward signal of scoring a goal to be too infrequent to properly train a state-based agent through reinforcement learning methods. Our agent consists of a Fully Convolutional Network (FCN) that predicts the location of the puck in the game environment, coupled with a controller that determines the steering of the karts based on the puck's location. While our agent fails to surpass the preexisting AI agents of PySTK, it is able to accurately determine the puck's location in most situations and succeeds in scoring goals against the AI agents. This project provided invaluable insights for future development in automated player agents for gaming environments.*

## 1. Introduction & Motivation

SuperTuxKart [1] is an open-source kart racing game similar to Mario Kart that is available on Mac, Linux, Windows, and Android. The PySuperTuxKart Python library, also referred to as PySTK, is based on SuperTuxKart and serves as a platform for artificial intelligence research. The objective of this project was to develop an automated agent capable of scoring as many goals as possible against other automated agents in two vs. two PySTK ice hockey matches.

Figure 1. Gameplay from PySTK ice hockey match.

We commenced by carefully considering the two potential approaches we could pursue for our agent: a state-based approach or a vision-based approach. The state-based approach relies on a deep network that receives the true state of the game, such as the location of the puck, the opposing team's karts, and the agent's karts, and outputs actions for each of the two agent's karts. These actions specify the steering and acceleration of the karts. The vision-based approach relies on a deep network to predict the location of key game objects, such as the puck and opposing team's karts, based on images from the perspective of our agent's karts. In addition, the true location of the agent's karts is provided in the vision-based approach. Features calculated based on these locations are then passed into a controller to determine the actions for our team's karts. The controller can either be completely custom designed or it can make use of a preexisting state-based agent designed for PySTK. While either approach could be utilized to create a robust ice hockey agent, we needed to select a single agent type to be eligible to compete in a tournament against other automated agents of the same type.

Thus, we evaluated both approaches based on their potential, their limitations, and our motivations for the project. We recognized that if we pursued a state-based approach, we would need to employ reinforcement learning methods to train our deep network. While reinforcement learning offers a powerful framework for learning decision-making policies in dynamic environments, highly complex environments such as PySuperTuxKart ice hockey present several challenges, including state representation, reward design, policy stability, sample efficiency, and generalization. Reinforcement learning approaches require a strong reward signal and we feared that given how infrequently goals are scored in PySTK ice hockey matches, the reward signal of scoring a goal would occur too infrequently to effectively train a state-based agent.

Although the vision-based agent has the disadvantage of not knowing the true state of the game, we believed that a well-trained object detection network would be able to accurately detect the location of key objects and reconstruct the game state from the provided gameplay images. In fact, because the state-based agent is not provided with the location of items and power-ups, the vision-based agent could in theory reconstruct a more detailed representation of the game state, as these objects are visible in the provided gameplay images. The vision-based approach also has the advantage of allowing a controller with manually designed components, in comparison to the state-based agent which must be a single deep network. By utilizing some manually designed controller logic, we believed a vision-based agent might be able to recover more quickly from situations in which the agent's karts get stuck as compared to the state-based agent. Finally and most importantly, as our research interests primarily lie in the domain of computer vision rather than reinforcement learning, we viewed the vision-based approach as a great opportunity to further our expertise in computer vision. For these reasons, our team decided to pursue a vision-based approach for our agent.

Before expounding on the design of our vision-based agent, it is important to briefly consider the place of this project within the larger context of computer vision research. The history of computer vision spans more than half a century. In 1963, Larry Roberts authored his PhD thesis titled "Machine Perception of Three-Dimensional Solids. [2]" The ambitious Summer Vision Project at MIT was initiated in 1966. David Marr's book, "Vision", published in the late 1960s, influenced a generation of computer vision scientists [3]. Throughout the 1960s to the 1980s, computer vision remained a challenging area for the research community. Around 1999 to 2000, machine learning techniques, including early neural networks, began permeating the computer vision domain. Simultaneously, the

internet's growth led to a surge in image data availability. Datasets such as PASCAL and ImageNet spurred algorithm development for object classification and detection using neural networks. The ImageNet Large Scale Visual Recognition Challenge [4] accelerated the advancement of deep learning-based computer vision algorithms, minimizing error rates. Although Convolutional Neural Networks (CNNs) were introduced in the 1990s, they experienced a breakthrough moment with AlexNet in 2012 [5]. Residual Networks (ResNets) [6] debuted in 2015, followed closely by U-Net architectures [7].

For the image processing component of our vision-based agent, we employ a deep neural network that combines the ResNet and U-Net architectures for optimal object detection and image segmentation. This combination represents a robust approach to image analysis tasks, harnessing ResNet's feature extraction capabilities and U-Net's precise segmentation abilities. For the controller component of our agent, we used both manually designed logic in conjunction with a preexisting PySTK state-based agent. Given the complexity of PySTK ice hockey games, we found it challenging to account for all of the possible gameplay situations within a completely custom controller. Thus, we used the preexisting controller as a foundation and utilized custom logic to enhance the performance of the preexisting controller in particularly challenging gameplay situations.

## 2. Methods

**Data collection:** The first step in implementing the object detection network of our agent was collecting a dataset of images and labels to train our network. Our objective was to maximize the number of goals scored by our agent so we recognized that the primary function of our object detection network should be to detect the screen coordinates of the center of the puck in the gameplay images. Thus, our first two labels for every training image were the X and Y screen coordinates of the puck in the image, with each coordinate being in the range -1 to 1. Although the screen coordinates of the puck can provide useful information to the controller, it is more valuable to determine the world coordinates of the puck within the game environment. Because our agent is provided with the intrinsic and extrinsic matrices of the camera used to generate the gameplay images, it is possible to use these matrices, along with the screen coordinates of the puck and the distance from the puck to the camera, to determine the world coordinates of the puck. Therefore, our third label for each image was the distance from the puck to the camera, which would allow us to derive the world coordinates of the puck.

To collect our training dataset of images and labels, we ran a game with the PySTK AI as the agent for both team one and team two. We used the built-in PySTK AI as our training agent because it performs strongly and provides a wide variety of possible actions, including hitting the opposing team's karts and using power-ups. Each ice hockey match is typically played until three goals are scored or 1,200 frames are played, but for the purpose of collecting training data, we played a game for 2,000 frames with no score limit. For each frame and for each of the four karts, we saved the gameplay image from the perspective of the kart as a JPG file. Along with each image, we saved a CSV file specifying the true X and Y screen coordinates of the puck in the image and the Euclidean distance between the puck center and the camera. We determined the true screen coordinates of the puck center based on the true puck world coordinates and the intrinsic and extrinsic matrices of the camera. We saved the image for all four players because we wanted our model to be able to identify the puck regardless of the team of the player (red or blue).

Our saved image files had a resolution of 400 x 300 pixels with three color channels (RGB) to correspond precisely with the images our agent

would be provided with during gameplay. We saved the images as JPG files rather than PNG files as JPG files have considerably smaller file sizes and do not distort the image enough to hinder the training of our objection detection network.

Because the puck is not always in the gameplay images, we needed to determine how to label the image when the puck was not visible. For the label of the puck screen coordinates, we assigned a value of (1, -1) if the puck was not in the image. These screen coordinates correspond to a point in the upper right corner of the image. The assigned X coordinate is arbitrary but we assigned a Y coordinate of -1 because this Y coordinate corresponds to the top of the image and the puck will never be located in the top region of the image. Thus, our model learned to predict a location in the top region of the image if the puck was not visible in the image. For the distance label, we assigned a label of 150 when the puck was not visible. The ice hockey map is only 129 units long so a distance label of 150 corresponds to a location outside the bounds of the map. The size of the puck in the image is inversely proportional to the distance between the puck and the camera so this label of 150 provides a simple means of teaching the model to maintain this inversely proportional relationship between the puck size and puck to camera distance.

To collect our validation set, we used the same approach as for the training set except this time we ran the match for only 1,000 frames. To ensure that the training and validation sets were sufficiently unique, we ran both the training and validation matches with random PySTK seeds. Upon manually inspecting all of our training and validation images and labels, we found that the puck was in the image about half of the time and not in the image about half of the time, resulting in a well-balanced dataset. We also found the labels for the puck screen coordinates and the puck to camera distance to be accurately assigned regardless of whether or not the puck was in the image.

**Model Architecture:** Having collected our training data, we then finalized the architecture for our object detection network. We used a 6-layer fully-connected neural network in a U-Net architecture with residual connections between three down-blocks and three corresponding up-blocks. Each down-block and up-block is composed of the following components:

1. 2D Convolution
2. Batch Normalization
3. ReLU
4. 2D Convolution
5. Batch Normalization
6 ReLU

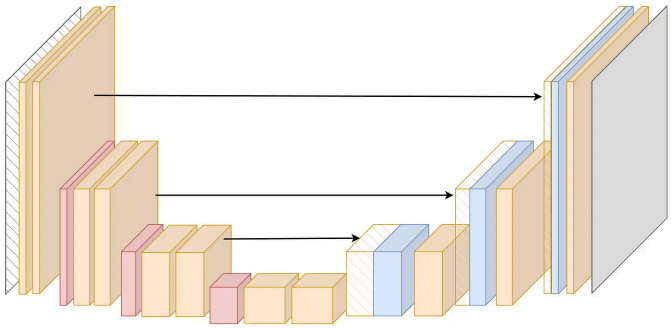

Figure 2. U-Net Architecture [8]

The first down-block receives 3 channels and outputs 16 channels, the second down-block receives 16 channels and outputs 32 channels, and the third down-block receives 32 channels and outputs 64 channels. A max-pooling layer is placed in between each down-block. The down-blocks reduce the spatial resolution of the input while increasing the number of channels.

This process is then reversed through three up-blocks that increase the spatial resolution to match that of the input image while decreasing the number of channels from 64, to 32, to 16. Each up-block is connected to its corresponding down-block via a residual connection. A 1x1 convolution then reduces the output to two channels. Finally, a spatial argmax is applied to the first channel of the output to produce the first two

logits used to teach the model to predict the X and Y coordinates of the puck. Global average pooling is applied to the second channel of the output to produce the logit used to teach the model to predict the puck to camera distance. The network ultimately produces three logits per input image.

**Model training:** To train our objection detection network, we used the MSE loss for the logits of both the puck screen coordinates and the puck to camera distance. Because the scale of the distance logits (0 to 150) was much larger than the scale of the screen coordinate logits (-1 to 1), we used a loss scaling hyperparameter with a value of 0.01 to reduce the effect the MSE loss of the distance logits had on the overall MSE loss.

We trained our model on a T4 GPU for 90 epochs with a learning rate of 0.001 and a batch size of 8. We used Adam as our optimizer and implemented a learning rate schedule to reduce the learning rate by a factor of 10 if the MSE on the validation set had not decreased in the last 15 epochs. We also implemented early stopping to save the model whenever a new minimum validation MSE loss value was achieved.

To reduce overfitting, we applied the random horizontal flip data augmentation to the images in our training set. Because the appearance of the ice hockey map and puck are constant, we found the color of the objects and background in the gameplay images to be a strong signal that could be used to identify the puck. Thus, we did not apply the color jitter data augmentation, observing that altering the color profile of the images reduced the performance of our model.

**Model Visualization:** While the MSE of our model on the validation set provided a baseline measure of the model's generalization performance, its usefulness in determining the model's performance during actual gameplay was limited. The MSE alone could not be used to determine if the model could predict the puck screen coordinates and distance accurately enough to provide useful information for the controller. Thus, to determine our model's performance during gameplay, we modified the function used to run PySTK ice hockey matches to visualize the model's predictions along with the ground truth labels when the "verbose" argument was used. Specifically, the predicted puck screen coordinates were overlaid onto the image as a green dot while the ground truth puck screen coordinates were displayed as a red dot. We also used text overlays to display the predicted and ground truth puck to camera distances.

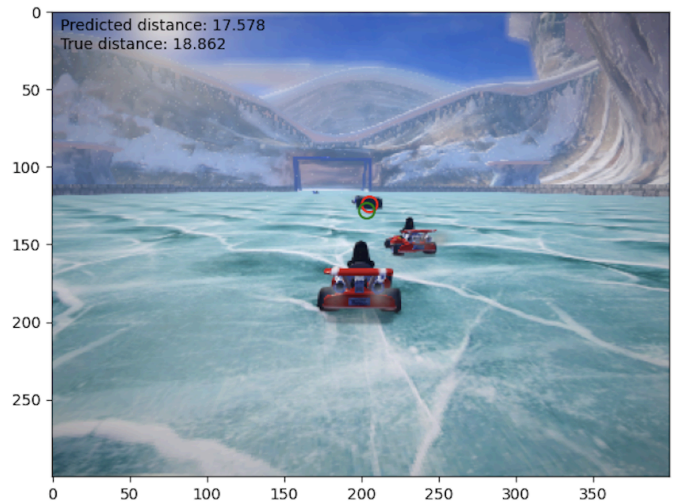


Figure 3. Visualization of predicted and ground-truth puck screen coordinates and distance.

Using this visualization method, we confirmed that the model nearly always predicted the correct X and Y screen coordinates of the puck when the puck was in the image and predicted a puck location in the top of the image (such as the sky or mountains) when the puck was not in the image. Furthermore, when the puck was relatively close to our agent's kart, our model was able to accurately predict the puck to camera distance. However, when the puck was relatively far away from our agent's kart, the predicted puck to camera distance deviated significantly from the ground truth value. Fortunately, this increased error in the predicted distance at long range did not have a

devastating impact on the performance of our controller.

The insights gained through this visualization method also influenced several key aspects of our controller. For example, we identified that a threshold Y coordinate value of -0.5 could be used to determine if the puck is or is not in the image. If the model predicts a Y coordinate value < -0.5, we can conclude that the puck is not in the image. Otherwise, we can conclude that the puck is in the image and use the predicted coordinates as the screen coordinates of the puck. We also recognized that if the puck was in front of the agent's kart but at a great distance away, the model would occasionally predict that the puck was not in the image. This is because at large distances, the puck is so small that it is not identifiable, even by a human observer. Thus, when we designed our controller, we kept in mind that the puck may be in front of the agent's kart even if the model predicts the puck is not in the image.

**Controller Design:** The puck screen coordinates and puck to camera distance predicted by our model were used to calculate several features that represented the current state of the game. These features were then used by our controller to determine the acceleration, steering, and braking of our agent's karts.

The most important of these features was the world coordinates of the puck, which represents the location of the puck within the game environment. The world coordinates of the puck were derived from the screen coordinates of the puck in the gameplay image, the distance from the puck to the camera that took the image, and the extrinsic and intrinsic matrices of the camera. Because the puck world coordinates are independent of the individual agent's karts, we were able to calculate the puck world coordinates if either of the karts were able to see the puck from their perspective. We considered the puck to be visible in a kart's gameplay image if the predicted Y screen coordinate of the puck was less than -0.5, otherwise we considered the puck to be not visible.

Other important features we utilized were the locations of the goals and the front and center of the agent's kart, which were provided during gameplay. From this information, we derived the direction of the agent's kart, the angle between the kart and the puck, and the distance between the puck and the goal line. Finally, we also used the velocity of the agent's kart, which was provided during gameplay.

We initially attempted to pass these features into a completely handcrafted controller but soon realized the tremendous difficulty with this approach. Due to the complex nature of the two vs. two PySTK ice hockey matches, we found it challenging to manually account for all of the possible situations the kart might find itself in during gameplay. Therefore, we ultimately decided to make use of a preexisting PySTK controller, named Image Jurgen Agent, that accepted as input the predicted features representing the game state and returned as output the kart actions. The Image Jurgen Agent relies on the world coordinates of the puck so when neither kart can detect the puck's location, we still use manual logic to determine the kart's actions.

To determine the action for one of our agent's karts, the high level strategy for our controller can be summarized as follows:

1. Use the provided gameplay image and kart state to extract features representing the current state of the game.
2. Determine if the kart is stuck, i.e., relatively stationary in the game environment.
3. If the kart is stuck, attempt to act in a way that gets the kart unstuck.
4. If the kart is not stuck, determine if the puck is visible from the perspective of the kart.

5. If the puck is visible, use the Image Jurgen Agent to engage in offense, i.e., try to score a goal.
6. If the puck is not visible, use manual logic to try to find the puck.

Because our primary objective was to score as many goals as possible, rather than to prevent the other team from scoring, we elected to not implement any defensive behavior and instead focus both of our karts on offensive actions. With both of our karts focused on moving the puck to the opposing goal, we hoped the opposing team would have difficulty gaining possession of the puck and scoring goals of their own. We found that the Image Jurgen Agent was able to successfully move the puck towards the opposing goal in most situations when the world coordinates of the puck were known, meaning at least one of the karts could see the puck.

When the puck is not visible by either of our agent's karts, both karts simply reverse and turn to search for the puck. The first kart turns clockwise while the second kart turns counterclockwise. When the puck becomes visible by at least one of the karts, the puck world coordinates are then calculated and the karts resume offensive behavior.

One issue we noticed with the offensive behavior of the Image Jurgen Agent was that the karts would tend to remain relatively stationary when the puck was moving with a low velocity near the karts. We referred to this situation as the kart being "stuck." To determine if a kart was stuck, we saved the location and velocity of the kart over the past 20 gameplay frames. We then deemed a kart to be stuck if the average velocity over these frames was less than 0.5 or the average location of the kart had not changed by more than a distance of 4 units. To get the kart "unstuck", we used similar logic as when looking for the puck. Specifically, the first kart reversed and turned clockwise for 10 frames while the second kart reversed and turned counter-clockwise for 10 frames. By forcing the kart to change its location and orientation, we hoped the kart would be able to escape the scenario that was causing it to get stuck and therefore be able to resume offense.

## 3. Results

We tested our agent on a local installation of PySTK against four opposing state-based agents. These opposing agents are named the Geoffrey, Jurgen, Yann, and Yoshua agents. We played 32 matches against each opposing agent, for a total of 128 matches. Each match is played for 1,200 frames or 3 goals scored by either team, whichever occurs first. Our agent scored an average of 0.68 goals per match and won 45 times, lost 39 times, and tied 44 times. The performance of our agent against the opposing AI agents is summarized in Figure 4 of the Appendix.

Our definition of a successful agent is one that scores at least 0.7 goals per match on average. As our agent did not meet this benchmark level of performance, we consider our agent unsuccessful. However, our agent still performs reasonably well as it won more matches than it lost against the opposing agents and its 0.68 goals scored per match is only slightly below the benchmark.

We believe the primary reason our agent failed to meet the benchmark for a successful project is because of the weakness of our controller. Despite our best efforts, the controller would often fail to quickly get our karts unstuck when they were caught in undesirable situations. Our controller also did not have the capability of directing our karts to play defense or use in-game items. Furthermore, our offensive strategy was highly simplistic and the karts would often push the puck along the side walls of the map rather than directly pushing the puck into the goal from the center of the map.

Unlike our controller, our objection detection FCN model performed fairly strongly. The model was able to correctly determine whether or not the puck was in the gameplay image 97.4% of the time. When the puck was in the image, the model predicted the X and Y screen coordinates of the center of the puck with an average error rate of 4.9%. However, our model did not perform as strongly when predicting the puck to camera distance. The predicted distance tended to deviate significantly from the true distance when the puck was relatively far from the karts, with an average error rate of 28.6%. From the learning curves presented in Figure 5 of the Appendix, we can see that while the FCN converges on a MSE loss of 0.16 on the training set, on the validation set it converges on a MSE loss of 0.75. As the MSE loss on the validation set is over four times larger than on the training set, the FCN is clearly overfitting to the training set. Despite these shortcomings, we consider the object detection model to be successful because it succeeds in providing the controller with valuable information about the game state.

## 4. Conclusion

This paper presented an automated, vision-based agent designed for the PySuperTuxKart (PySTK) ice hockey game environment, highlighting an application of computer vision within interactive games. Our agent utilized a Fully Convolutional Network (FCN) object detection model to reconstruct the game state from gameplay images and a controller to determine actions for our agent's karts based on the reconstructed game state. Despite our agent not meeting our benchmark of 0.7 goals scored per match, the vision-based system successfully detected the puck's location in most scenarios and managed to score an average of 0.68 goals per match against the opposing AI agents.

In the future, we would like to explore several alternative methods to try to improve the performance of our agent. First, rather than implementing a vision-based agent, we could try implementing a fully state-based agent using reinforcement learning techniques. Although the ultimate objective of our agent is to score as many goals as possible, goals are scored relatively infrequently so scoring a goal would likely be a poor reward signal. Instead, we could try training our agent to minimize the distance of the puck to the opposing goal line, which could serve as an appropriate proxy for scoring a goal. To collect training data, we would save the state of the game at each frame of a match along with the corresponding actions performed by a high performing AI agent or human player. We would then train our agent to imitate this AI or human expert to achieve a baseline level of performance, and then improve this performance further using policy gradient or gradient free optimization methods.

Even if we continued to implement a vision-based agent, there are several additional strategies we could employ to attempt to improve our agent's performance. For example, as there are 18 unique kart types in PySTK, each with their own size and driving characteristics, we could treat the kart type as another hyperparameter and test our agent with each kart to find the one resulting in the best performance. We might also be able to improve the puck to camera distance predictions of our model by collecting more training data, adding more layers or channels to our FCN, or using a different loss function such as L1 loss rather than MSE loss during training. Our controller might also be improved by using items and power ups to impede the progress of opposing karts, adding logic to prevent our karts from colliding with each other, and manually adding logic to override the suggested actions of our Image Jurgen Agent when those actions are clearly counterproductive, such as moving the puck towards our own goal. We could also try making use of another pre-existing PySTK controller that incorporates information about the opposing karts to play defense, which would also

require us to modify our FCN to predict the location of the opposing karts.

Based on the results of this project, we conclude that agents employing vision-based models are capable of accurately reconstructing the state of their surrounding environment. However, no matter how accurately the vision-based model reconstructs the environment, a robust controller is necessary to determine how the agent should act in a vast multitude of possible scenarios. These findings illustrate why developing a physical, fully self-driving car is so challenging. Modern advancements in sensor technology and computer vision techniques allow self-driving cars to accurately determine the location of objects around them. But given the enormous complexity of driving in metropolitan areas, there are an incalculable number of situations the car must learn to navigate in. With the consequences of an error so high, the performance of self-driving cars must be virtually perfect in order for them to gain widespread adoption. We hope that with further advancements in reinforcement learning techniques, it will be possible to efficiently create robust controllers for a variety of complex tasks, including playing PySTK ice hockey matches and driving through the winding streets of downtown Seattle.

## References


[1] SuperTuxKart Development Team. "SuperTuxKart." Available at: https://supertuxkart.net/, Accessed on March 20, 2024.

[2] Larry Roberts. "Machine Perception of Three-Dimensional Solids." PhD thesis, Massachusetts Institute of Technology, 1963.

[3] David Marr. "Vision." MIT Press, 1982.

[4] Fei-Fei Li, Jia Deng, Li Fei-Fei, and Olga Russakovsky. "ImageNet Large Scale Visual Recognition Challenge." International Journal of Computer Vision, vol. 115, no. 3, pp. 211-252, 2015.

[5] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. "ImageNet Classification with Deep Convolutional Neural Networks." In Proceedings of the 25th International Conference on Neural Information Processing Systems (NIPS), 2012.

[6] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. "Deep Residual Learning for Image Recognition." In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.

[7] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. "U-Net: Convolutional Networks for Biomedical Image Segmentation." In Proceedings of the International Conference on Medical Image Computing and Computer-Assisted Intervention (MICCAI), 2015

[8] Conor O'Sullivan. "U-Net Explained: Understanding Its Image Segmentation Architecture." Towards Data Science, 2023, towardsdatascience.com/u-net-explained -understanding-its-image-segmentation-architecture -56e4842e313a.

## Appendix

| Opposing agent | Matches played | Average goals scored per match | Matches won | Match lost | Matches tied |
|---|---|---|---|---|---|
| Geoffrey | 32 | 0.84 | 16 | 8 | 8 |
| Jurgen | 32 | 0.84 | 14 | 12 | 6 |
| Yann | 32 | 0.59 | 9 | 15 | 8 |
| Yoshua | 32 | 0.44 | 6 | 4 | 22 |
| *Total* | 128 | 0.68 | 45 | 39 | 44 |

Figure 4. Performance of our agent against the opposing PySTK agents as measured by the average goals scored per match by our agent and the number of matches won, lost, and tied.

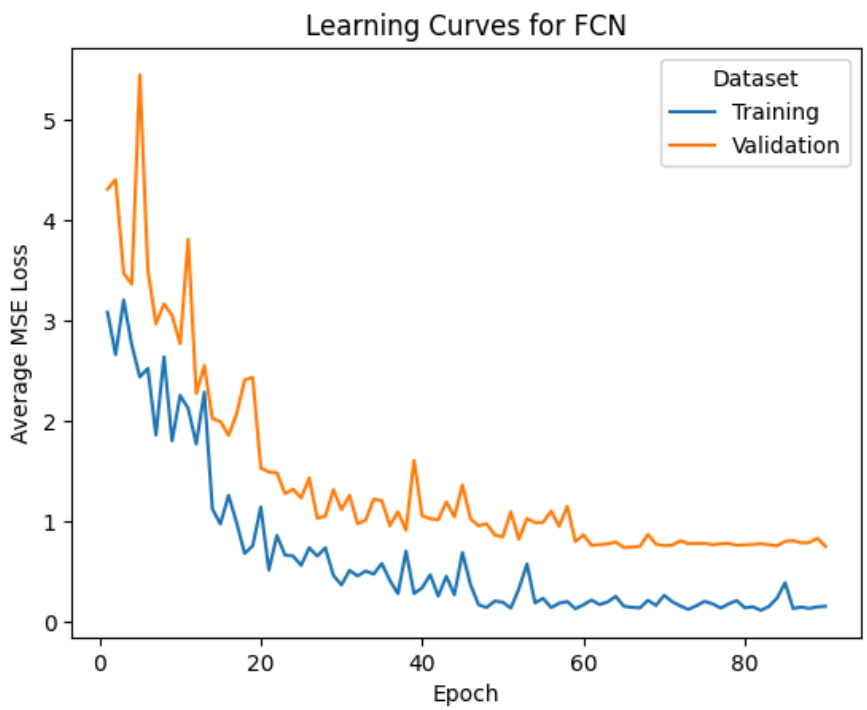


Figure 5. Learning curves for FCN objection detection network showing average MSE loss for each epoch of the Adam optimization process. The MSE loss was calculated as: (MSE loss of predicted puck screen coordinates) + 0.01(MSE loss of predicted puck to camera distance).